

If we include the component in our *AppController*, it will be available in all the other controllers and there is no need to include it in each controller file.

For example, to use our *PasswordComponent* and its *generate()* method in *UserController*, issue the following code:

```
App::uses('AppController', 'Controller');

class UserController extends AppController {

    public $components = array('Session', 'Password');

    public function some_action() {

        $password = $this->Password->generate(12);

    }

}
```

We can use this generated password in the view file by setting the view variable or saving it to the database.



Note: In the latest version of CakePHP, which is 3.0, loading of a component is a little different. There is a method called *loadComponent()* to load any component in the controller. So, to include our password component, we can use *\$this->loadComponent('Password')*. After this, we can use the component in the same way as we do in CakePHP 2.0: *\$this->Password->generate()*.

Using other components in our component

We can use other components—built-in or custom—inside our own component. CakePHP provides an easy way to do this. Similar to controllers, we can define other components in the *\$components* property of our component.

For example, if there is an existing component named *ExampleComponent*, and we want to use it in our *PasswordComponent*, we can issue the following code:

```
App::uses('Component', 'Controller');
class PasswordComponent extends Component {
    public $components = array('Example');
    public function some_function_name() {
        $foo = $this->Example->bar();
    }
}
```

```
}
/* other code goes here*/
}
```

As you can see, we can use another component and its methods in the same way using *\$this->Component->method()*.

Magic methods of a component

CakePHP provides some callback functions known as magic methods for components. We can use these callback methods to fulfil our needs. These methods are automatically called by CakePHP, so we can define the logic that needs to be performed before or after some action or event.

Given below is a list of the available magic methods.

initialize (Controller \$controller): This method is called before the controller's *beforeFilter* method. We can use this method to initialise our variables or objects and use them later.

startup (Controller \$controller): This method is called after the controller's *beforeFilter* method, but before the controller executes its current action handler. This method can also be used to carry out some processes before our main action executes, like checking a session for a logged-in user, or setting cookies, etc.

beforeRender (Controller \$controller): This method is called after the controller executes the requested action's logic, but before the controller renders its views and layout. Here we can set the view variables, so that we can use them in our view/layout files.

shutdown (Controller \$controller): This method is called before the output is sent to the browser. We can use it to unset the unnecessary variables and objects to free up the memory.

beforeRedirect (Controller \$controller, \$url, \$status=null, \$exit=true): This method is called when the controller's redirect method is called but before any further action is taken. In this method, we can check some conditions and reset the redirect URL or disable the redirection from here. 

By: Narendra Vaghela

The author works as a tech lead in an IT company. He can be contacted at narendravaghela4389@gmail.com.



For More
Test and Equipment Stories
Visit www.electronicssb2b.com